You said:

codigo.c
Code

codigo.h
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 38s

ChatGPT said:

Análise estática de malware — `codigo.c` e `codigo.h`

Veredito preliminar

Malicioso — rootkit para Linux

Classificação baseada no código-fonte dos dois arquivos enviados.

O código implementa mecanismos para ocultar processos, esconder o próprio módulo do kernel e obter privilégios de root por meio de chamadas especiais ao sistema. Esses comportamentos são característicos de um **rootkit de kernel (LKM rootkit)**.

O indício mais conclusivo está no próprio arquivo `codigo.c`, que declara a descrição `LKM rootkit` e implementa funções de ocultação e elevação de privilégios. O cabeçalho `codigo.h` define os identificadores e os sinais especiais usados para acionar essas funcionalidades.

A análise a seguir é estática: avalia o que o código foi escrito para fazer, sem executar ou instalar o módulo.

1. Identificação dos arquivos

Os arquivos formam um conjunto de código C para Linux:

- `codigo.c`: implementa as funções principais do módulo de kernel.
- `codigo.h`: declara funções, constantes, identificadores de sinais e estruturas necessárias à implementação.

O próprio `codigo.c`, nas linhas 493–498, registra os pontos de inicialização e encerramento do módulo e informa a descrição `LKM rootkit`. codigo.c

A denominação LKM significa *Loadable Kernel Module*, ou módulo carregável do kernel. Isso permite que o código seja integrado diretamente ao núcleo do sistema operacional, caso seja compilado e carregado.

2. Análise dos principais comportamentos

Etapa 1 — Descoberta e alteração da tabela de chamadas do sistema

A função `get_syscall_table_bf()` procura o endereço da tabela de chamadas do sistema, conhecida como `sys_call_table`. O código também contém uma função chamada `resolve_sym()`, utilizada para resolver símbolos internos do kernel. codigo.c

Essa capacidade permite modificar a forma como determinadas operações do sistema são processadas.

Avaliação: comportamento de alto risco, pois possibilita interceptar operações fundamentais do Linux.

Etapa 2 — Ocultação de arquivos e processos

As funções `hacked_getdents()` e `hacked_getdents64()` interceptam chamadas usadas para obter entradas de diretórios.

O código examina os nomes encontrados e remove da listagem:

- Arquivos e diretórios cujo nome começa com o prefixo `diamorphine_secret`.
- Processos identificados por seu PID, quando estão marcados como invisíveis e aparecem nas listagens de `/proc`.

O prefixo é declarado em `codigo.h`:

```
#define MAGIC_PREFIX "diamorphine_secret"
```

A lógica correspondente está implementada nas duas funções de interceptação. codigo.h codigo.c codigo.c

Avaliação: funcionalidade explícita de ocultação de artefatos, característica de rootkits. Os arquivos ocultados não precisam ser excluídos do disco: podem simplesmente deixar de aparecer nas listagens fornecidas às aplicações.

Etapa 3 — Ativação de comandos especiais por sinais

A função `hacked_kill()` intercepta a chamada de sistema `kill()` e interpreta determinados números de sinal como comandos especiais.

O cabeçalho `codigo.h` define:

Sinal	Comportamento
`31 — SIGINVIS`	Alterna a invisibilidade do processo indicado pelo PID.
`63 — SIGMODINVIS`	Alterna a visibilidade do módulo do kernel.
`64 — SIGSUPER`	Aciona a função de elevação de privilégios.

Essas constantes estão definidas diretamente no cabeçalho. codigo.h

A implementação que interpreta os sinais está na função `hacked_kill()`. codigo.c

Avaliação: o uso de sinais para ativar essas funções cria uma interface de controle oculta, sem a necessidade de um arquivo de comando visível para cada operação.

Etapa 4 — Elevação de privilégios para root

A função `give_root()` prepara novas credenciais e altera os identificadores de usuário e grupo, incluindo UID e EUID, para zero. Em Linux, o identificador zero corresponde ao usuário root.

A alteração é aplicada por meio de `prepare_creds()` e `commit_creds()`. codigo.c

Quando `hacked_kill()` recebe o sinal especial `64`, chama essa função. codigo.c

Avaliação: isso implementa diretamente uma capacidade de elevação de privilégios. Um processo que acione com sucesso esse mecanismo poderá adquirir credenciais de root, dependendo das condições de execução e compatibilidade do módulo.

Etapa 5 — Ocultação do próprio módulo

As funções `module_hide()` e `module_show()` modificam a lista de módulos do kernel:

- `module_hide()` remove o módulo da lista.
- `module_show()` reinsere o módulo na lista.

Além disso, durante a inicialização, `diamorphine_init()` chama `module_hide()` e `tidy()`. A segunda função libera e zera os atributos de seção do módulo. codigo.c codigo.c

Avaliação: a ocultação deliberada da própria presença é uma característica central de um rootkit.

Etapa 6 — Modificação de áreas protegidas da memória do kernel

O código implementa `unprotect_memory()` e `protect_memory()` para modificar as permissões de memória.

Nas arquiteturas x86, altera o controle de escrita do registrador CR0; no ARM64, utiliza uma função para modificar permissões de mapeamento. Em seguida, a inicialização instala interceptações nas chamadas `getdents`, `getdents64` e `kill`. codigo.c codigo.c

Isso permite alterar o comportamento do sistema operacional em nível de kernel, em vez de atuar apenas como um programa comum.

Avaliação: comportamento consistente com adulteração do kernel e evasão de mecanismos de inspeção.

3. Comparação com referências externas

Os comportamentos encontrados correspondem aos descritos publicamente para o **Diamorphine**, um rootkit de kernel para Linux. A análise publicada pelo Avast Threat Labs descreve a ocultação do módulo, de processos e de arquivos, além da elevação de privilégios por sinais especiais. 【turn925759search0】

A documentação do Wazuh também utiliza o Diamorphine como exemplo de rootkit capaz de esconder processos e de modificar a visibilidade do módulo. 【turn925759search1】

A semelhança não depende apenas do nome: ela aparece na combinação específica de funções de ocultação, interceptação de chamadas de sistema, sinais especiais e alteração de credenciais encontrada nos arquivos enviados.

4. O que esta análise permite concluir — e o que não permite

O código apresenta evidências fortes de três capacidades maliciosas:

- **Evasão e ocultação:** esconder processos, arquivos, diretórios e o próprio módulo.
- **Elevação de privilégios:** alterar as credenciais para obter privilégios de root.

- **Manipulação do kernel:** interceptar chamadas de sistema e modificar proteções de memória.

Por outro lado, os arquivos fornecidos, por si só, não demonstram comunicação com um servidor remoto, roubo de dados, execução remota de comandos ou persistência após uma reinicialização. Também não comprovam que o módulo tenha sido compilado ou carregado em algum equipamento.

Essas limitações não tornam o código benigno; apenas delimitam o que é possível afirmar com base na evidência disponível.

Conclusão final

Classificação: MALWARE

Categoria principal: Rootkit de kernel para Linux (*Linux Kernel Loadable Module Rootkit*).

Confiança da classificação: Alta.

Justificativa: o código implementa intencionalmente ocultação de processos e arquivos, ocultação do próprio módulo, interceptação de chamadas do sistema e elevação de privilégios. O conjunto de funcionalidades é consistente com o rootkit Diamorphine.

Decisão: os arquivos `codigo.c` e `codigo.h` devem ser classificados como código-fonte de um rootkit, e não como código benigno de uso comum. O fato de o código poder ser utilizado em pesquisas ou testes autorizados não elimina seu potencial malicioso. A classificação do código, entretanto, não equivale à confirmação de que um sistema esteja infectado.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Rootkit de kernel para Linux (Linux Kernel Rootkit)**, especificamente do tipo **LKM (Loadable Kernel Module)**.

Ele atua no nível do kernel para ocultar processos, arquivos e o próprio módulo, além de permitir a elevação de privilégios para root. O código analisado é compatível com o rootkit **Diamorphine**.